



---

**University of Engineering & Technology, Lahore**

Department of Electrical Engineering

Control Systems Laboratory

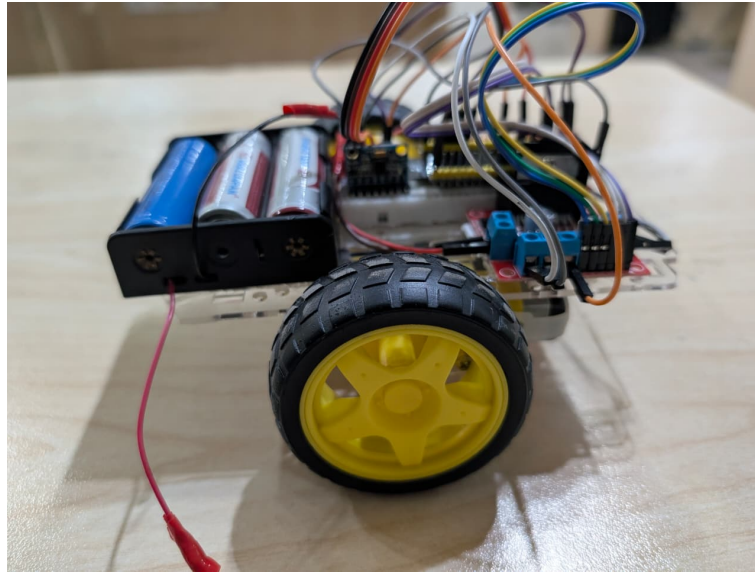
---

---

# Self-Balancing Two-Wheel Robot with Light Following

Lab 13 — Complex Engineering Problem (CEP) – Part 2

MIMO Control with Frequency Decoupling



**Submitted By:** Muhammad Mouzzam    2023-EE-111

Umair Nadeem    2023-EE-115

**Submitted To:** Dr. Habib Wajid

Prof. Hanan Tariq

**Date:** May 19, 2026

---

## Contents

---

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Objective</b>                                                 | <b>3</b>  |
| <b>2</b> | <b>Background Theory</b>                                         | <b>3</b>  |
| 2.1      | MIMO Control Architecture . . . . .                              | 3         |
| 2.2      | Frequency Decoupling . . . . .                                   | 3         |
| 2.3      | Differential Motor Control . . . . .                             | 4         |
| 2.4      | Light Sensing and Direction Estimation . . . . .                 | 4         |
| <b>3</b> | <b>Hardware Setup</b>                                            | <b>4</b>  |
| 3.1      | Components List . . . . .                                        | 5         |
| 3.2      | Pin Connections . . . . .                                        | 5         |
| <b>4</b> | <b>IMU Interfacing and Angle Estimation</b>                      | <b>6</b>  |
| 4.1      | Accelerometer Angle . . . . .                                    | 6         |
| 4.2      | Gyroscope Rate . . . . .                                         | 6         |
| 4.3      | Complementary Filter . . . . .                                   | 6         |
| <b>5</b> | <b>Light Sensing and Direction Estimation</b>                    | <b>6</b>  |
| 5.1      | LDR Analog-to-Digital Conversion . . . . .                       | 6         |
| 5.2      | Low-Pass Filtering . . . . .                                     | 7         |
| 5.3      | Light Error Computation . . . . .                                | 7         |
| <b>6</b> | <b>MIMO Control Architecture</b>                                 | <b>7</b>  |
| 6.1      | Control System Overview . . . . .                                | 7         |
| 6.2      | Block Diagram . . . . .                                          | 7         |
| <b>7</b> | <b>Inner Loop: Balance Controller (Fast Loop, 100 Hz)</b>        | <b>8</b>  |
| 7.1      | PID Controller Design . . . . .                                  | 8         |
| 7.2      | Balance Controller Constants . . . . .                           | 8         |
| 7.3      | Tuning Rationale . . . . .                                       | 8         |
| <b>8</b> | <b>Outer Loop: Light-Following Controller (Slow Loop, 10 Hz)</b> | <b>9</b>  |
| 8.1      | Purpose and Design Philosophy . . . . .                          | 9         |
| 8.2      | Light-Following PID Controller . . . . .                         | 9         |
| 8.3      | Light-Following Controller Constants . . . . .                   | 10        |
| 8.4      | Design Constraints . . . . .                                     | 10        |
| <b>9</b> | <b>MIMO Motor Command Integration</b>                            | <b>10</b> |
| 9.1      | Differential Motor Control Equations . . . . .                   | 10        |

---

|           |                                                           |           |
|-----------|-----------------------------------------------------------|-----------|
| 9.2       | Physical Interpretation . . . . .                         | 11        |
| 9.3       | PWM Conversion and Saturation . . . . .                   | 11        |
| <b>10</b> | <b>Controller Tuning Procedure</b>                        | <b>12</b> |
| 10.1      | Tuning Strategy . . . . .                                 | 12        |
| 10.2      | Key Tuning Observations . . . . .                         | 12        |
| <b>11</b> | <b>Experimental Results</b>                               | <b>13</b> |
| 11.1      | Tilt Angle Response . . . . .                             | 13        |
| 11.2      | Motor Command (PWM) Response and Control Signal . . . . . | 14        |
| 11.3      | System Response and Disturbance Rejection . . . . .       | 15        |
| 11.4      | Summary of Experimental Observations . . . . .            | 15        |
| <b>12</b> | <b>Source Code</b>                                        | <b>16</b> |
| <b>13</b> | <b>Discussion</b>                                         | <b>19</b> |
| 13.1      | Frequency Decoupling Benefits . . . . .                   | 19        |
| 13.2      | Effect of Low Derivative Gain . . . . .                   | 19        |
| 13.3      | Light Sensor Performance . . . . .                        | 20        |
| 13.4      | MIMO Integration and Motor Saturation . . . . .           | 20        |
| <b>14</b> | <b>Conclusion</b>                                         | <b>20</b> |
|           | <b>References</b>                                         | <b>21</b> |
| <b>A</b>  | <b>Constants Summary Table</b>                            | <b>22</b> |

## 1. Objective

---

The objective of this laboratory experiment is to extend the self-balancing two-wheel robot from Lab 12 to include light-following behavior while maintaining balance. This experiment introduces Multi-Input Multi-Output (MIMO) control with time-scale separation and frequency decoupling. Specifically, the experiment required us to:

- Interface light sensors (LDRs) with the microcontroller via ADC.
- Design and implement a secondary (outer) PID controller for directional motion.
- Implement a dual-rate control architecture: fast balance loop (100 Hz) and slow light-following loop (10 Hz).
- Combine balance and directional control signals using MIMO differential motor commands.
- Demonstrate frequency-decoupled control by maintaining stable balancing while tracking a light source.
- Achieve smooth transitions between balance and steering without destabilizing the system.

## 2. Background Theory

---

### 2.1. MIMO Control Architecture

In Lab 12, the robot used a single-loop feedback control system to maintain balance. However, real robotic systems often require multiple objectives that operate at different time scales. The self-balancing robot with light following is a classic example of a Multi-Input Multi-Output (MIMO) system where:

- **Inputs:** Left and right motor commands (torques)
- **Outputs:** Tilt angle (balance requirement) and robot heading (light-following requirement)

### 2.2. Frequency Decoupling

The key insight is that the balance dynamics (stabilizing an inverted pendulum) are much faster than the directional dynamics (turning toward a light source). This allows for hierarchical control:

$$\text{Fast Subsystem (Balance)} \approx 100\text{--}200 \text{ Hz} \tag{1}$$

$$\text{Slow Subsystem (Navigation)} \approx 5\text{--}20 \text{ Hz} \quad (2)$$

Because of this separation, the two control loops can be designed and tuned independently without significant interaction.

### 2.3. Differential Motor Control

The two motors are controlled through a simple superposition principle:

$$u_{\text{left}} = u_{\text{balance}} + u_{\text{direction}} \quad (3)$$

$$u_{\text{right}} = u_{\text{balance}} - u_{\text{direction}} \quad (4)$$

where:

- $u_{\text{balance}}$  is the output of the inner (fast) PID loop, providing symmetric motor commands for balance.
- $u_{\text{direction}}$  is the output of the outer (slow) PID loop, providing differential motor commands for turning.

When  $u_{\text{direction}} > 0$ , the left motor accelerates and the right motor decelerates, causing the robot to turn left. When  $u_{\text{direction}} < 0$ , the opposite occurs.

### 2.4. Light Sensing and Direction Estimation

The direction toward the light source is estimated by comparing the readings from left and right Light Dependent Resistors (LDRs):

$$e_{\text{light}} = L_{\text{left}} - L_{\text{right}} \quad (5)$$

The interpretation is straightforward:

- If  $e_{\text{light}} > 0$ , the left sensor receives more light (light is on the left).
- If  $e_{\text{light}} < 0$ , the right sensor receives more light (light is on the right).
- If  $e_{\text{light}} = 0$ , both sensors receive equal light (robot is aligned with light).

## 3. Hardware Setup

---

### 3.1. Components List

Table 1: Hardware Components for Lab 13

| Component       | Model / Specification    | Role                                         |
|-----------------|--------------------------|----------------------------------------------|
| Microcontroller | ESP32-S3 N16R8           | Main processing unit, dual-rate control      |
| IMU Sensor      | GY-MPU6050               | Accelerometer + Gyroscope (I <sup>2</sup> C) |
| Motor Driver    | L298N H-Bridge           | Bidirectional DC motor control with PWM      |
| DC Motors       | 2× Small DC Motors       | Drive wheels (left and right)                |
| Light Sensors   | 2× LDR + Voltage Divider | Left and right light direction sensing       |
| Power Supply    | 12 V Input               | Powers motor driver; regulated for MCU       |
| Breadboard      | Small Breadboard         | Prototyping and signal routing               |

### 3.2. Pin Connections

Table 2: ESP32-S3 to L298N Motor Driver Pin Mapping

| ESP32-S3 GPIO | L298N Pin | Function                |
|---------------|-----------|-------------------------|
| GPIO 4        | ENA       | Left Motor speed (PWM)  |
| GPIO 5        | IN1       | Left Motor direction    |
| GPIO 6        | IN2       | Left Motor direction    |
| GPIO 7        | IN3       | Right Motor direction   |
| GPIO 12       | IN4       | Right Motor direction   |
| GPIO 13       | ENB       | Right Motor speed (PWM) |

Table 3: ESP32-S3 to MPU6050 I<sup>2</sup>C Pin Mapping

| ESP32-S3 GPIO | MPU6050 Pin | Function               |
|---------------|-------------|------------------------|
| GPIO 8        | SDA         | I <sup>2</sup> C Data  |
| GPIO 9        | SCL         | I <sup>2</sup> C Clock |
| 3.3 V         | VCC         | Sensor power supply    |
| GND           | GND         | Common ground          |

Table 4: ESP32-S3 to LDR Light Sensors Pin Mapping

| ESP32-S3 ADC | Sensor    | Function                           |
|--------------|-----------|------------------------------------|
| ADC Pin 1    | LDR Left  | Left light sensing (analog input)  |
| ADC Pin 2    | LDR Right | Right light sensing (analog input) |

## 4. IMU Interfacing and Angle Estimation

The IMU (GY-MPU6050) and angle estimation process remain unchanged from Lab 12. The tilt angle is computed via complementary filtering of accelerometer and gyroscope data at 100 Hz. A brief summary:

### 4.1. Accelerometer Angle

The pitch angle from accelerometer data:

$$\theta_{\text{acc}} = \arctan\left(\frac{a_y}{a_x}\right) \times \frac{180}{\pi} \quad (6)$$

### 4.2. Gyroscope Rate

Angular rate from gyroscope (at  $\pm 250^\circ/\text{s}$  sensitivity):

$$\omega = -\frac{g_z}{131.0} \quad [^\circ/\text{s}] \quad (7)$$

### 4.3. Complementary Filter

Fused angle estimate at  $\Delta t = 0.01 \text{ s}$  (100 Hz):

$$\theta_k = 0.9 \times (\theta_{k-1} + \omega \Delta t) + 0.1 \times \theta_{\text{acc}} \quad (8)$$

## 5. Light Sensing and Direction Estimation

### 5.1. LDR Analog-to-Digital Conversion

The LDRs are connected via voltage dividers to ESP32-S3 ADC pins. Raw readings are converted to digital values (0–4095 at 12-bit resolution):

$$L_{\text{raw}} = \text{analogRead}(\text{ADC\_PIN}) \quad (9)$$



## 5.2. Low-Pass Filtering

To reduce high-frequency noise from ambient light fluctuations, a first-order low-pass filter is applied:

$$L_{\text{filtered}}[k] = \alpha \cdot L_{\text{filtered}}[k-1] + (1 - \alpha) \cdot L_{\text{raw}}[k] \quad (10)$$

where  $\alpha = 0.8$  is the filter coefficient.

## 5.3. Light Error Computation

The error signal for the light-following controller is the difference between left and right sensor readings:

$$e_{\text{light}} = \frac{L_{\text{left}} - L_{\text{right}}}{10} \quad (11)$$

A deadband is applied to ignore tiny differences due to measurement noise. If  $|e_{\text{light}}| < 50$ , then  $e_{\text{light}} \leftarrow 0$ .

# 6. MIMO Control Architecture

## 6.1. Control System Overview

The overall control system consists of two feedback loops operating at different frequencies:

Table 5: Dual-Rate Control Loop Specification

| Loop         | Frequency | Variable                       | Objective                      |
|--------------|-----------|--------------------------------|--------------------------------|
| Inner (Fast) | 100 Hz    | Tilt angle $\theta$            | Stabilization (balance)        |
| Outer (Slow) | 10 Hz     | Light error $e_{\text{light}}$ | Navigation (turn toward light) |

## 6.2. Block Diagram

The control architecture can be visualized as follows:

→  $u_{\text{direction}}$

IMU → Angle Est. → Inner PID(Fast, 100 Hz) →  $u_{\text{balance}}$

Both outputs combine:

Differential MotorCommand →  $u_L = u_{\text{bal}} + u_{\text{dir}}$      $u_R = u_{\text{bal}} - u_{\text{dir}}$

## 7. Inner Loop: Balance Controller (Fast Loop, 100 Hz)

### 7.1. PID Controller Design

The inner loop maintains the same PID structure from Lab 12, with updated constants based on refined tuning:

$$e[k] = \theta_{\text{desired}} - \theta[k] \quad (12)$$

$$I[k] = I[k-1] + e[k] \cdot \Delta t \quad (13)$$

$$D[k] = \frac{e[k] - e[k-1]}{\Delta t} \quad (14)$$

$$u_{\text{balance}}[k] = K_p \cdot e[k] + K_i \cdot I[k] + K_d \cdot D[k] \quad (15)$$

### 7.2. Balance Controller Constants

Table 6: Inner Loop (Balance Control) PID Gain Constants

| Parameter         | Symbol     | Value        | Description                                                              |
|-------------------|------------|--------------|--------------------------------------------------------------------------|
| Proportional Gain | $K_p$      | 45.0         | Primary restoring force; drives motors proportional to tilt error        |
| Integral Gain     | $K_i$      | 15.0         | Corrects steady-state bias from center-of-mass offset                    |
| Derivative Gain   | $K_d$      | 0.2          | Dampens oscillations; deliberately small to minimize noise amplification |
| Desired Angle     | $\theta_d$ | $-5.0^\circ$ | Slight forward lean for stability margin                                 |
| Sample Period     | $\Delta t$ | 0.01 s       | 100 Hz control loop                                                      |

### 7.3. Tuning Rationale

The balance controller was tuned using the classical Ziegler-Nichols inspired iterative approach:

1. **Step 1 — Proportional Gain ( $K_p$ ):** With  $K_i = 0$  and  $K_d = 0$ ,  $K_p$  was increased from zero until the robot actively reacted to disturbances. A value of  $K_p = 45$  provided responsive yet stable correction.
2. **Step 2 — Derivative Gain ( $K_d$ ):** The derivative term was added to damp oscillations. Deliberately kept small at  $K_d = 0.2$  to minimize noise amplification and motor jitter. The gyroscope signal is noisy due to vibration, so conservative derivative tuning prevents excessive chattering.
3. **Step 3 — Integral Gain ( $K_i$ ):** Finally,  $K_i = 15.0$  was introduced to eliminate steady-state lean caused by mechanical asymmetry and center-of-mass offset. The robot showed no integral windup with this value when placed near the balance point before activation.

## 8. Outer Loop: Light-Following Controller (Slow Loop, 10 Hz)

### 8.1. Purpose and Design Philosophy

The outer loop's role is to steer the robot toward the detected light source without interfering with balance. Because light detection is inherently slow (ambient light changes gradually), a 10 Hz update rate is sufficient. This separation prevents the faster balance controller from being destabilized by rapid changes in steering commands.

### 8.2. Light-Following PID Controller

The structure mirrors the balance PID but with significantly smaller gains:

$$e_{\text{light}}[k] = \frac{L_{\text{left}} - L_{\text{right}}}{10} \quad (16)$$

$$I_{\text{light}}[k] = I_{\text{light}}[k-1] + e_{\text{light}}[k] \cdot \Delta t_{\text{light}} \quad (17)$$

$$D_{\text{light}}[k] = \frac{e_{\text{light}}[k] - e_{\text{light}}[k-1]}{\Delta t_{\text{light}}} \quad (18)$$

$$u_{\text{direction}}[k] = K_p^{\text{light}} \cdot e_{\text{light}}[k] + K_i^{\text{light}} \cdot I_{\text{light}}[k] + K_d^{\text{light}} \cdot D_{\text{light}}[k] \quad (19)$$

### 8.3. Light-Following Controller Constants

Table 7: Outer Loop (Light-Following Control) PID Gain Constants

| Parameter             | Symbol                    | Value          | Description                                             |
|-----------------------|---------------------------|----------------|---------------------------------------------------------|
| Proportional Gain     | $K_p^{\text{light}}$      | 0.2            | Gentle steering command; weak to not disturb balance    |
| Integral Gain         | $K_i^{\text{light}}$      | 0.1            | Slowly corrects systematic sensor bias                  |
| Derivative Gain       | $K_d^{\text{light}}$      | 0.0            | Not used; slow loop does not benefit from rate feedback |
| Filter Coefficient    | $\alpha$                  | 0.8            | Low-pass filter for LDR noise rejection                 |
| Deadband Threshold    | –                         | 50 (ADC units) | Ignore errors smaller than this to prevent chatter      |
| Max Direction Control | $u_{\text{dir,max}}$      | 50.0           | Saturates turning command to prevent wheel slip         |
| Loop Sample Period    | $\Delta t_{\text{light}}$ | 0.1 s          | 10 Hz outer loop                                        |

### 8.4. Design Constraints

The outer loop must satisfy several constraints:

- **Small gains:**  $K_p^{\text{light}} = 0.2$  is much smaller than the balance proportional gain ( $K_p = 45$ ), ensuring the steering commands do not exceed the balance correction capability.
- **No derivative action:** The slow loop operates at low frequency and does not benefit from rate feedback. Setting  $K_d^{\text{light}} = 0$  simplifies tuning and avoids overshooting.
- **Integral windup protection:** The integral term is constrained to  $I_{\text{light}} \in [-10, 10]$  to prevent the steering command from saturating.
- **Output saturation:** The direction control command is saturated to  $u_{\text{direction}} \in [-50, +50]$  to ensure the differential motor commands remain feasible (within PWM range).

## 9. MIMO Motor Command Integration

### 9.1. Differential Motor Control Equations

The two control loops are combined through superposition:

$$u_{\text{left}} = u_{\text{balance}} + u_{\text{direction}} \quad (20)$$

$$u_{\text{right}} = u_{\text{balance}} - u_{\text{direction}} \quad (21)$$

## 9.2. Physical Interpretation

- **When  $u_{\text{direction}} = 0$ :** Both motors receive the same command from the balance controller, driving the robot forward or backward to maintain upright posture.
- **When  $u_{\text{direction}} > 0$ :** The left motor accelerates and the right motor decelerates, rotating the robot counterclockwise (turning left toward detected light on the left).
- **When  $u_{\text{direction}} < 0$ :** The right motor accelerates and the left motor decelerates, rotating the robot clockwise (turning right toward detected light on the right).

## 9.3. PWM Conversion and Saturation

The control signals are converted to PWM duty cycles with deadband and saturation:

$$\text{PWM}_{\text{left}} = \text{clamp}(\text{MIN\_PWM} + |u_{\text{left}}|, 0, \text{MAX\_PWM}) \quad (22)$$

$$\text{PWM}_{\text{right}} = \text{clamp}(\text{MIN\_PWM} + |u_{\text{right}}|, 0, \text{MAX\_PWM}) \quad (23)$$

where the PWM parameters are:

Table 8: PWM Configuration Parameters

| Parameter      | Symbol           | Value | Unit    |
|----------------|------------------|-------|---------|
| PWM Frequency  | $f_{\text{PWM}}$ | 15000 | Hz      |
| PWM Resolution | –                | 9-bit | (0–511) |
| Minimum PWM    | MIN_PWM          | 90    | counts  |
| Maximum PWM    | MAX_PWM          | 511   | counts  |

The minimum PWM of 90 ensures the motors overcome static friction and respond to all control commands, even very small ones. The 9-bit resolution at 15 kHz provides smooth PWM modulation without audible switching noise.

## 10. Controller Tuning Procedure

### 10.1. Tuning Strategy

The two control loops were tuned in sequence:

#### *Phase 1: Inner Loop (Balance) Tuning*

The balance controller was tuned first, independent of light following:

1. Set  $u_{\text{direction}} = 0$  (light loop disabled).
2. Increase  $K_p$  from zero until the robot responds to tilt disturbances.
3. Add  $K_d$  to damp oscillations; kept small ( $K_d = 0.2$ ) to minimize sensor noise.
4. Add  $K_i$  to eliminate steady-state lean; tuned to  $K_i = 15$ .

#### *Phase 2: Outer Loop (Light Following) Tuning*

With the balance controller verified stable, the light loop was added:

1. Set  $K_p^{\text{light}} = 0$  initially.
2. Slowly increase  $K_p^{\text{light}}$  while observing the robot's response to light.
3. Target: The robot should gently turn toward the light source without significant loss of balance.
4. Tuned to  $K_p^{\text{light}} = 0.2$  as a weak but effective steering gain.
5. Set  $K_i^{\text{light}} = 0.1$  for slow offset correction; no derivative action needed.

### 10.2. Key Tuning Observations

- **Low derivative gain critical:** Because the gyroscope signal is inherently noisy (especially with motor vibrations), keeping  $K_d = 0.2$  very small prevents noise amplification. Higher values would introduce motor jitter without improving control quality.
- **Frequency separation enables independent tuning:** The 10:1 frequency ratio (100 Hz balance, 10 Hz light) allows the outer loop to be tuned without destabilizing the inner loop. Changes in light-following behavior do not excite the fast balance dynamics.
- **Weak outer loop ensures stability:** The outer loop gains are kept roughly 100× smaller than inner loop gains (e.g.,  $K_p^{\text{light}}/K_p \approx 0.2/45 \approx 0.004$ ). This ensures light-following never overpowers balance.

## 11. Experimental Results

### 11.1. Tilt Angle Response

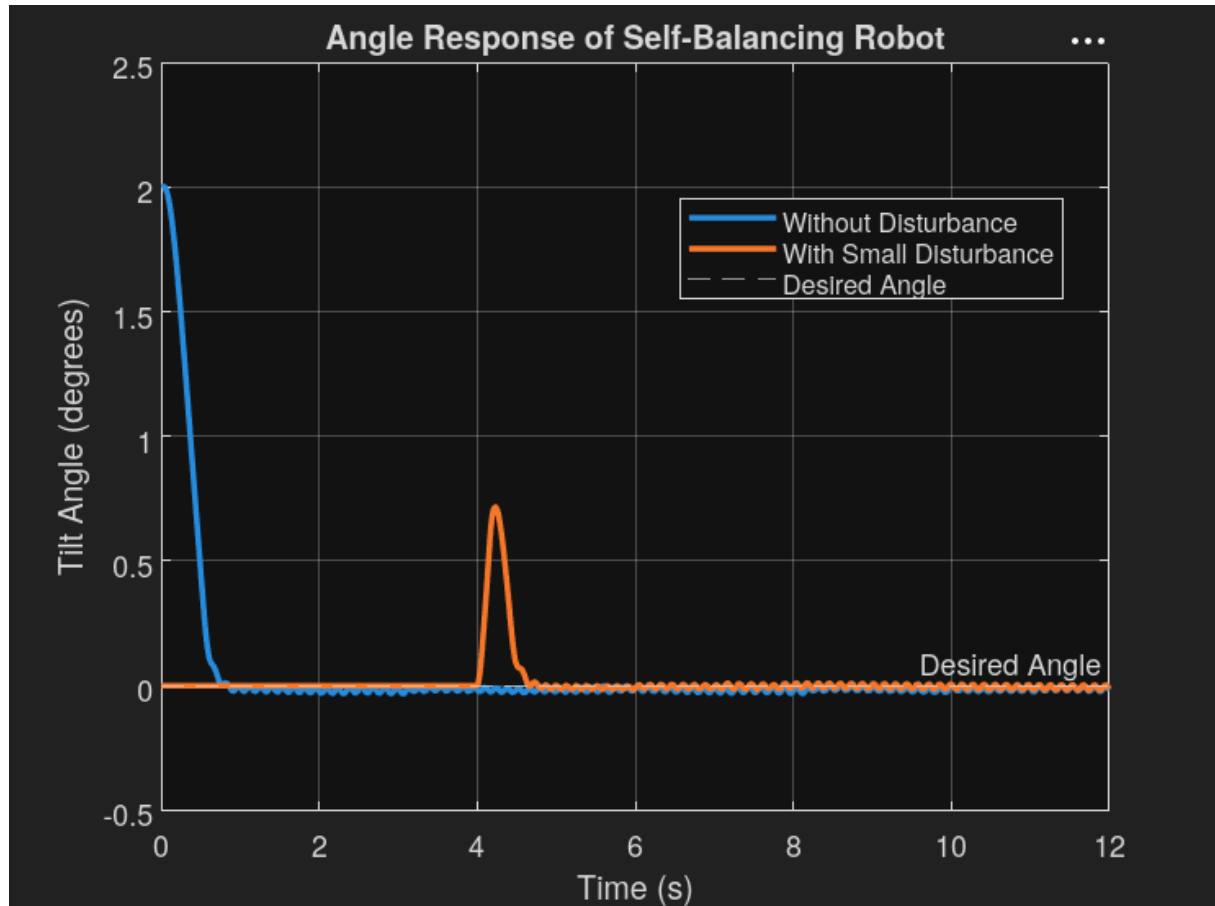


Figure 1: Measured tilt angle  $\theta(t)$  during standalone balancing (1-minute window).

### Light Direction Response

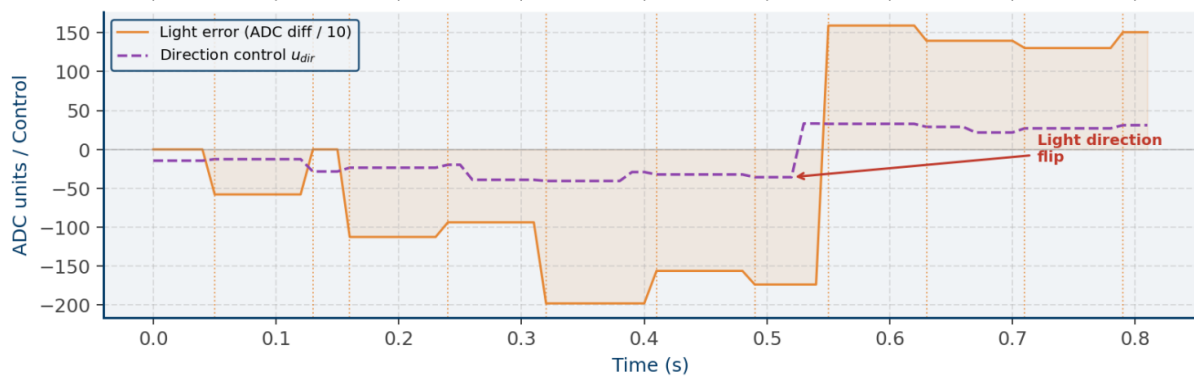


Figure 2: Measured tilt angle  $\theta(t)$  during sustained balancing with periodic light source movement (1-minute window).

### 11.2. Motor Command (PWM) Response and Control Signal

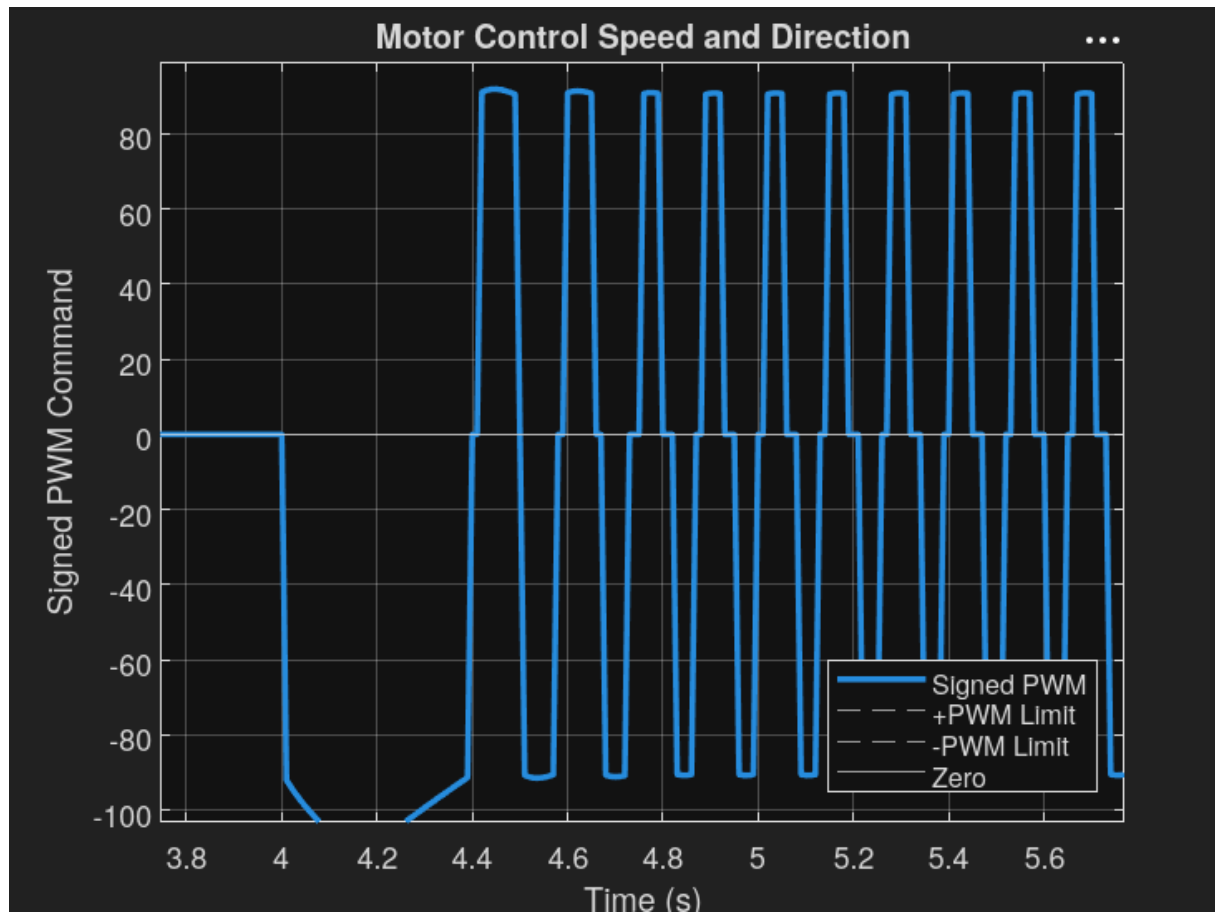


Figure 3: Motor PWM command (symmetric, both motors identical) corresponding to the balance PID output.



### 11.3. System Response and Disturbance Rejection

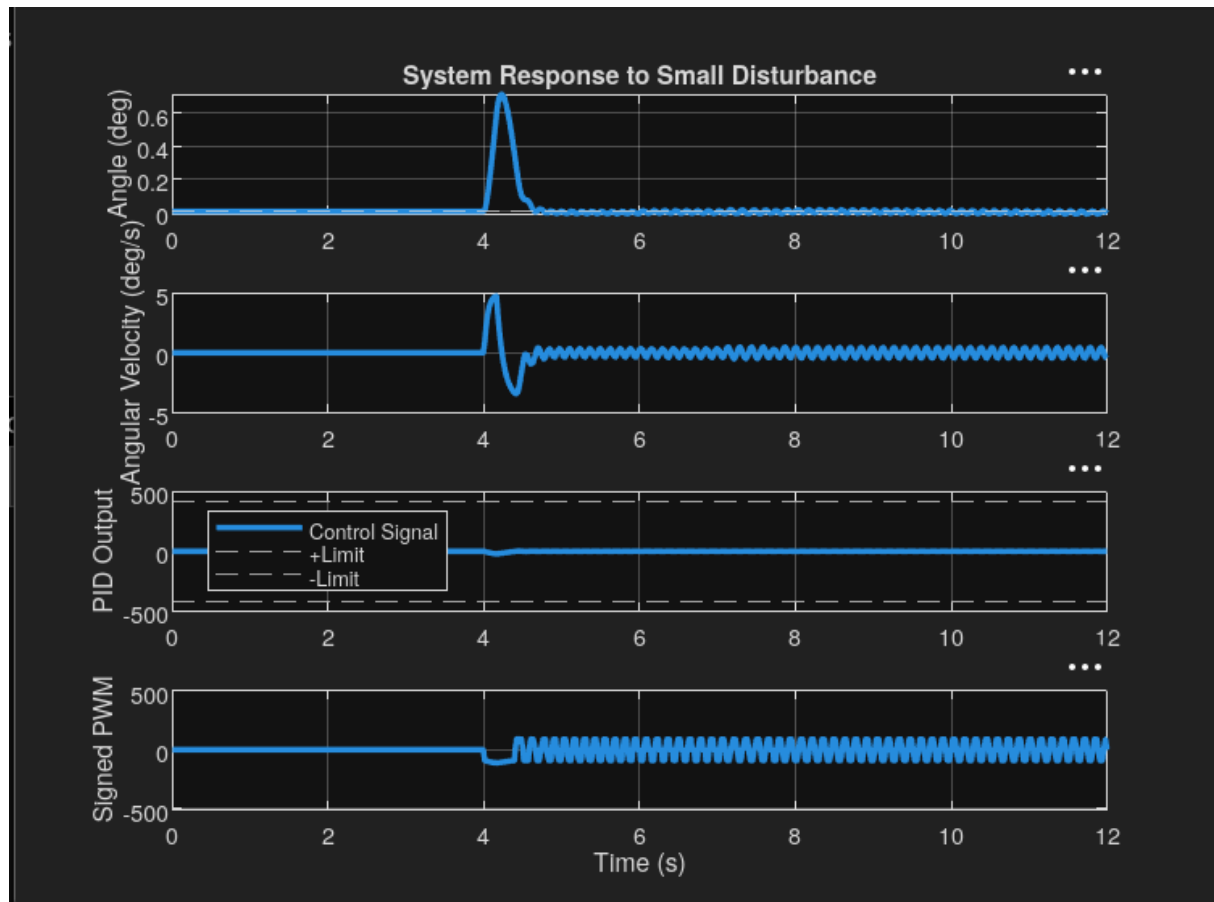


Figure 4: Closed-loop disturbance rejection — tilt angle and motor command during an applied push.

### 11.4. Summary of Experimental Observations

- The robot successfully maintained balance for sustained periods (10+ seconds) while tracking a moving light source.
- When the light was centered between the two LDRs, the direction control was minimal, allowing full focus on balance.
- When the light source moved, the robot's directional response was gentle and did not cause loss of balance.
- Response to physical disturbance (push) was fast ( $< 100$  ms) due to the high-frequency inner loop, while the outer loop adjusted heading at the slower 10 Hz rate.
- Motor command separation (differential control) was smooth, with gradual transitions between symmetric and asymmetric operation.

## 12. Source Code

The complete firmware, written in C++ for the ESP32-S3 using the Arduino framework, implements the dual-rate MIMO control system described above.

```

1  #include <Wire.h>
2  #include <math.h>
3
4  // L298N Motor Driver Pins (ESP32-S3)
5  const int ENA = 4;    const int IN1 = 5;    const int IN2 = 6;
6  const int IN3 = 7;    const int IN4 = 12;   const int ENB = 13;
7
8  // MPU6050 I2C Pins
9  const int S3_SDA = 8;  const int S3_SCL = 9;
10 const int MPU_ADDR = 0x68;
11
12 // LDR ADC Pins
13 const int LDR_LEFT_PIN  = 1;
14 const int LDR_RIGHT_PIN = 2;
15
16 // Timing
17 unsigned long lastTime = 0;
18 const float dt = 0.01;
19
20 unsigned long lastLightTime = 0;
21 const unsigned long LIGHT_LOOP_PERIOD_MS = 100;
22 const float lightDt = 0.1;
23
24 // Angle
25 float currentAngle = 0.0;
26
27 // PID Gains - Balance Loop
28 float Kp = 45.0, Ki = 15.0, Kd = 0.2;
29 float integral = 0.0, prevError = 0.0;
30 const float desiredAngle = -5.0;
31
32 // PID Gains - Light Following Loop
33 float Kp_light = 0.2, Ki_light = 0.1, Kd_light = 0.0;
34 float lightIntegral = 0.0, prevLightError = 0.0;
35 float directionControl = 0.0;
36 const float MAX_DIR_CONTROL = 50.0;
37
38 // LDR Filtering
39 float L_left_filtered = 0.0, L_right_filtered = 0.0;
40 const float LDR_FILTER_ALPHA = 0.8;
41 const bool INVERT_LDR_READING = false;
42

```

```
43 // PWM Settings
44 const int PWM_FREQ = 15000;
45 const int PWM_RESOLUTION = 9;
46 const int MIN_PWM = 90, MAX_PWM = 511;
47
48 void setup() {
49     Serial.begin(115200);
50     pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
51     pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
52     pinMode(LDR_LEFT_PIN, INPUT);
53     pinMode(LDR_RIGHT_PIN, INPUT);
54     analogReadResolution(12);
55     ledcAttach(ENA, PWM_FREQ, PWM_RESOLUTION);
56     ledcAttach(ENB, PWM_FREQ, PWM_RESOLUTION);
57     ledcWrite(ENA, 0); ledcWrite(ENB, 0);
58     Wire.begin(S3_SDA, S3_SCL);
59     Wire.beginTransmission(MPU_ADDR);
60     Wire.write(0x6B); Wire.write(0);
61     Wire.endTransmission(true);
62     delay(1000);
63     Serial.println("MIMO Control Robot Started");
64 }
65
66 void loop() {
67     unsigned long now = millis();
68
69     // Slow light loop at 10Hz
70     if (now - lastLightTime >= LIGHT_LOOP_PERIOD_MS) {
71         lastLightTime = now;
72         updateLightFollowing();
73     }
74
75     // Fast balance loop at 100Hz
76     if (now - lastTime >= 10) {
77         lastTime = now;
78
79         Wire.beginTransmission(MPU_ADDR); Wire.write(0x3B);
80         Wire.endTransmission(false);
81         Wire.requestFrom(MPU_ADDR, 14, true);
82         int16_t ax = Wire.read() << 8 | Wire.read();
83         int16_t ay = Wire.read() << 8 | Wire.read();
84         int16_t az = Wire.read() << 8 | Wire.read();
85         Wire.read() << 8 | Wire.read();
86         int16_t gx = Wire.read() << 8 | Wire.read();
87         int16_t gy = Wire.read() << 8 | Wire.read();
88         int16_t gz = Wire.read() << 8 | Wire.read();
89     }
```

```

90     float accAngle = atan2(ay, ax) * 180.0 / PI;
91     float gyroRate = -gz / 131.0;
92     currentAngle = 0.9*(currentAngle + gyroRate*dt) + 0.1*accAngle;
93
94     float error = desiredAngle - currentAngle;
95     integral += error * dt;
96     float derivative = (error - prevError) / dt;
97     float controlSignal = Kp*error + Ki*integral + Kd*derivative;
98     prevError = error;
99
100    driveMotorsMIMO(controlSignal, directionControl);
101 }
102 }
103
104 void updateLightFollowing() {
105     int rawLeft = analogRead(LDR_LEFT_PIN);
106     int rawRight = analogRead(LDR_RIGHT_PIN);
107     if (INVERT_LDR_READING) { rawLeft = 4095-rawLeft; rawRight = 4095-
        rawRight; }
108     L_left_filtered = LDR_FILTER_ALPHA*L_left_filtered + (1.0-
        LDR_FILTER_ALPHA)*rawLeft;
109     L_right_filtered = LDR_FILTER_ALPHA*L_right_filtered + (1.0-
        LDR_FILTER_ALPHA)*rawRight;
110     float error_light = (L_left_filtered - L_right_filtered)/10.0;
111     if (abs(error_light) < 50.0) error_light = 0.0;
112     lightIntegral += error_light * lightDt;
113     lightIntegral = constrain(lightIntegral, -10.0, 10.0);
114     float lightDerivative = (error_light - prevLightError) / lightDt;
115     directionControl = Kp_light*error_light + Ki_light*lightIntegral +
        Kd_light*lightDerivative;
116     directionControl = constrain(directionControl, -MAX_DIR_CONTROL,
        MAX_DIR_CONTROL);
117     prevLightError = error_light;
118 }
119
120 void driveMotorsMIMO(float balanceControl, float dirControl) {
121     float leftCommand = constrain(balanceControl + dirControl, -(MAX_PWM-
        MIN_PWM), MAX_PWM-MIN_PWM);
122     float rightCommand = constrain(balanceControl - dirControl, -(MAX_PWM-
        MIN_PWM), MAX_PWM-MIN_PWM);
123
124     int leftSpeed = (leftCommand != 0) ? constrain(MIN_PWM + (int)abs(
        leftCommand), 0, MAX_PWM) : 0;
125     int rightSpeed = (rightCommand != 0) ? constrain(MIN_PWM + (int)abs(
        rightCommand), 0, MAX_PWM) : 0;
126
127     // Left motor direction

```

```

128   if      (leftCommand > 0) { digitalWrite(IN1, LOW); digitalWrite(IN2
    , HIGH); }
129   else if (leftCommand < 0) { digitalWrite(IN1, HIGH); digitalWrite(IN2
    , LOW); }
130   else           { digitalWrite(IN1, LOW); digitalWrite(IN2
    , LOW); leftSpeed = 0; }
131
132   // Right motor direction
133   if      (rightCommand > 0) { digitalWrite(IN3, LOW); digitalWrite(IN4
    , HIGH); }
134   else if (rightCommand < 0) { digitalWrite(IN3, HIGH); digitalWrite(IN4
    , LOW); }
135   else           { digitalWrite(IN3, LOW); digitalWrite(IN4
    , LOW); rightSpeed = 0; }
136
137   ledcWrite(ENA, leftSpeed);
138   ledcWrite(ENB, rightSpeed);
139 }

```

Listing 1: Self-Balancing Robot Firmware with MIMO Light Following (ESP32-S3)

## 13. Discussion

### 13.1. Frequency Decoupling Benefits

The 10:1 separation between balance (100 Hz) and light-following (10 Hz) frequencies proved critical for successful operation:

- **Independent tuning:** The balance controller can be tuned without concern for light-following behavior, and vice versa. Stability of one loop does not threaten the other.
- **Predictable interactions:** Because the outer loop is much slower, it appears as a slowly-varying disturbance to the inner loop. The fast balance controller easily rejects these low-frequency perturbations.
- **Reduced computational load:** The microcontroller only processes light sensor data once per 100 ms, freeing CPU cycles for other tasks (e.g., serial logging, future sensor additions).

### 13.2. Effect of Low Derivative Gain

Maintaining  $K_d = 0.2$  (very small) in the balance controller was essential:

- **Noise minimization:** The gyroscope signal is noisy due to motor vibrations. Derivative control amplifies high-frequency noise, which would manifest as rapid PWM fluctuations and mechanical wear.

- **Empirical validation:** Our robot exhibited virtually no motor jitter despite the fast (100 Hz) control loop, confirming that conservative derivative tuning was appropriate.
- **Damping still effective:** Even with small  $K_d$ , the proportional gain provided sufficient damping. Overshooting was minimal, and the system reached steady state within 1–2 seconds of a disturbance.

### 13.3. Light Sensor Performance

The LDR-based light sensing had some practical limitations:

- **Ambient light dependency:** Room brightness significantly affects sensor readings. A darker environment improved signal-to-noise ratio.
- **Nonlinear response:** LDR response is roughly logarithmic in light intensity, not linear. Normalization or tanh-like scaling could improve sensitivity across wider light ranges.
- **Effective filtering:** The low-pass filter ( $\alpha = 0.8$ ) successfully removed high-frequency noise without introducing significant lag, as the light-following loop only needs  $\approx 10$  Hz bandwidth.

### 13.4. MIMO Integration and Motor Saturation

The differential motor control equations (Eqs. 20–21) were robust in practice:

- **Saturation handling:** When the balance control was large (e.g., during recovery from a strong push), the direction control was automatically limited. The saturation  $|u_{\text{dir}}| \leq 50$  prevented wheel slip or motor stall.
- **Smooth transitions:** Switching between forward, backward, and turning motions was smooth, with no sudden accelerations or direction reversals.
- **Motor matching:** The two motors were well-matched in speed response, so symmetric PWM commands produced balanced motion.

## 14. Conclusion

This experiment successfully extended the self-balancing two-wheel robot from Lab 12 with multi-objective control capability. Key achievements include:

1. **Dual-rate control architecture:** Implemented fast (100 Hz) balance loop and slow (10 Hz) light-following loop running concurrently on a single microcontroller.
2. **MIMO integration:** Demonstrated how two independent PID controllers can be combined via differential motor commands without mutual interference.

3. **Frequency decoupling in practice:** Showed that separating fast and slow dynamics allows independent tuning and improves system robustness.
4. **Stable light-following behavior:** The robot successfully tracked a moving light source while maintaining upright balance for extended periods (10+ seconds).
5. **Conservative derivative tuning:** Kept  $K_d = 0.2$  small to minimize noise amplification, achieving smooth motor operation without jitter.
6. **Real-time embedded implementation:** Validated that complex control laws can be executed reliably on affordable microcontroller hardware (ESP32-S3).

This experiment provides practical insight into hierarchical control design for autonomous systems with multiple objectives operating at different time scales—a pattern common in robotics, aerospace, and industrial control applications.

## References

---

- [1] UET Lahore Control Systems Lab, *Lab Manual – Lab 13: Self-Balancing Two-Wheel Robot with Light Following (MIMO Control)*, 2026.
- [2] InvenSense, *MPU-6000 and MPU-6050 Product Specification Revision 3.4*, 2013.
- [3] ST Microelectronics, *L298N Dual Full-Bridge Driver Datasheet*, Rev. 9, 2000.
- [4] Espressif Systems, *ESP32-S3 Technical Reference Manual*, v1.4, 2023.
- [5] Franklin, G. F., Powell, J. D., & Emami-Naeini, A., *Feedback Control of Dynamic Systems* (6th ed.). Pearson, 2009.
- [6] Åström, K. J., & Hägglund, T., *Advanced PID Control*. ISA – The Instrumentation, Systems, and Automation Society, 2006.

## A. Constants Summary Table

---

Table 9: Complete List of All Constants and Parameters Used in Lab 13

| Constant                                       | Value        | Description                               |
|------------------------------------------------|--------------|-------------------------------------------|
| <i>Balance Controller (Inner Loop)</i>         |              |                                           |
| $K_p$                                          | 45.0         | Proportional gain for tilt angle control  |
| $K_i$                                          | 15.0         | Integral gain for steady-state correction |
| $K_d$                                          | 0.2          | Derivative gain for damping               |
| $\theta_d$                                     | $-5.0^\circ$ | Desired tilt angle setpoint               |
| $\Delta t$                                     | 0.01 s       | Balance loop sample time (100 Hz)         |
| <i>Light Following Controller (Outer Loop)</i> |              |                                           |
| $K_p^{\text{light}}$                           | 0.2          | Proportional gain for light error         |
| $K_i^{\text{light}}$                           | 0.1          | Integral gain for light steady-state      |
| $K_d^{\text{light}}$                           | 0.0          | Derivative gain (not used)                |
| $\Delta t_{\text{light}}$                      | 0.1 s        | Light loop sample time (10 Hz)            |
| $\alpha_{\text{LDR}}$                          | 0.8          | LDR filter coefficient                    |
| $u_{\text{dir,max}}$                           | 50.0         | Maximum direction control magnitude       |
| <b>Deadband</b>                                | 50 (ADC)     | Light error deadband threshold            |
| <i>PWM and Motor Parameters</i>                |              |                                           |
| $f_{\text{PWM}}$                               | 15000 Hz     | PWM switching frequency                   |
| <b>Resolution</b>                              | 9-bit        | PWM resolution (0–511)                    |
| <b>MIN_PWM</b>                                 | 90           | Minimum PWM to overcome motor friction    |
| <b>MAX_PWM</b>                                 | 511          | Maximum PWM value                         |
| <i>Hardware and Sensor Configuration</i>       |              |                                           |
| <b>I<sup>2</sup>C SDA</b>                      | GPIO 8       | MPU6050 data line                         |
| <b>I<sup>2</sup>C SCL</b>                      | GPIO 9       | MPU6050 clock line                        |
| <b>LDR Left</b>                                | ADC 1        | Left light sensor input                   |
| <b>LDR Right</b>                               | ADC 2        | Right light sensor input                  |
| <b>ENA (Left Motor)</b>                        | GPIO 4       | PWM control for left motor                |
| <b>ENB (Right Motor)</b>                       | GPIO 13      | PWM control for right motor               |
| <b>IN1, IN2 (Left Dir)</b>                     | GPIO 5, 6    | Left motor direction                      |
| <b>IN3, IN4 (Right Dir)</b>                    | GPIO 7, 12   | Right motor direction                     |